

Forge: Designing a More Capable Lean Tactic

Abstract

Forge is a design for a proof-producing automation layer above **Lean**'s `grind`, not a replacement for its congruence closure and existing theory solvers. Its objective is to close goals that require a change of proof strategy: discovering an ordered-algebra certificate, proving a missing side condition, generalizing an accumulator before induction, or constructing a witness. The central object is a scoped graph of proof obligations whose nodes are discharged by cooperating, independently checked engines.

The design specifies an acceptance boundary, algorithms, data structures, solver adapters, resource policies, failure semantics, and an evaluation protocol. Five mechanisms are prototyped in Python: finite polynomial-cone search, exact rational quadratic decomposition, adaptive Bernstein certificates, structural induction with accumulator generalization, and demand-sliced ground-Horn reasoning. The supplied run passes 1,680 assertions and exports 24 replayable certificate bundles. These are exact-arithmetic and proof-replay experiments in Python, not a verified Python kernel and not a performance comparison with `grind`. Generated **Lean** proofs and integration examples are supplied, but were not compiled in the experiment environment. The evidence supports the mechanisms and exposes their limitations; it does not yet establish broad superiority on real **Lean** projects.

Contents

1	Objective and current baseline	3
2	Related tools and the implementation position	4
3	User-facing behavior and success semantics	6
4	Scoped facts and proof obligations	7
5	The acceptance boundary and trust policy	8
6	Nonlinear ordered algebra: exact certificates	9
7	Bernstein certificates on bounded domains	12
8	Goal-directed theorem use and demand slicing	15
9	Induction, generalization, and invariant construction	16
10	Witnesses, quantified reasoning, and finite search	18
11	Cooperative scheduling and integration with grind	20
12	A worked cross-engine proof plan	21
13	Prototype implementation and measured results	22
14	A credible evaluation protocol	24
15	Implementation roadmap and explicit engineering tasks	25
16	Failure modes and safeguards	27
17	Conclusion	28
A	Package contents and reproduction	28
B	Certificate schema and replay invariants	29
C	Selected implementation excerpts	30
	Sources	32

1 Objective and current baseline

1.1 What “more powerful” should mean

There are three distinct claims one might make about a stronger tactic. A tactic can recognize more mathematical fragments; it can automate more proof strategies within the same logic; or it can solve more goals under a specified resource budget. These claims are not interchangeable. A procedure for polynomial inequalities expands a theory fragment, automatic induction expands proof strategy, and a better scheduler changes practical success at a budget. None changes the theorems accepted by *Lean*’s kernel.

The practical target is a substantial increase in *kernel-accepted, unattended proofs*, with explicit trust and resource policies. The design should preserve the existing successful path, discover missing intermediate statements, combine local solvers rather than merely run them in isolation, and provide a small reproducible explanation of every success. It should also say exactly why it stopped. “The external solver returned unsatisfiable” and “all goals were proved” must remain different outcomes.

The source snapshot is *Lean* 4.34.0, checked against the current reference manual and the release metadata on September 14, 2026, Pacific time. The accompanying example project pins Mathlib commit 1cf325a0cf67aca2b04d76b5380ff6a9e410aefa; its toolchain file specifies the same *Lean* version. This is a source-compatibility target, not a claim that the supplied examples compiled.¹²

1.2 What grind already does

The current grind is a substantial SMT-inspired system. It combines congruence closure, propositional propagation, controlled E-matching, case analysis, integer arithmetic, and algebraic solvers, with ordinary *Lean* proof terms for derived facts. Its standard-library annotations already provide a significant theorem supply. The manual also identifies large combinatorial Boolean search as a workload better routed to a SAT-based tactic.³

The algebraic component is not merely a ring normalizer. It uses polynomial rewriting and completion techniques related to Gröbner-basis computation, including reasoning over suitable commutative rings and fields. Adding another polynomial-equality engine is therefore not, by itself, a compelling improvement.⁴ Ordered linear reasoning is also already present and parameterized by algebraic structure.⁵

It is equally important not to misdescribe the control infrastructure. The E-matching interface already supports directional patterns, instantiation restrictions, and bounded generations; it is not a naive exhaustive enumeration of substitutions.⁶ The v4.34.0 Action source explicitly supports non-chronological backtracking and extraction of the useful tactic steps.⁷

A recent system description documents the plugin interface, premise-selection facilities, diagnostics, and opportunities for sharing state across verification conditions. These are existing assets to reuse, not inventions to claim for *Forge*.⁸

¹Lean FRO, release v4.34.0. Full reference: [1].

²Mathlib, pinned commit and toolchain file. Full reference: [2].

³Lean FRO, *The grind tactic*. Full reference: [3].

⁴Lean FRO, *Algebraic Solver (Commutative Rings, Fields)*. Full reference: [4].

⁵Lean FRO, *Linear Arithmetic Solver*. Full reference: [5].

⁶Lean FRO, *E-matching*. Full reference: [6].

⁷Lean source, *Lean.Meta.Tactic.Grind.Action*, tag v4.34.0. Full reference: [7].

⁸Kim Morrison and Leonardo de Moura, *grind: An SMT-Inspired Tactic for Lean 4*, IJCAR 2026. Full reference: [8].

1.3 The proposed increment

The design concentrates on three gaps between saturation and general proof construction. First, a solver may need a mathematically useful intermediate fact that is not already in its search vocabulary. Second, a goal may require an induction motive or witness, not just additional consequences of ground facts. Third, useful computations may have cheap checkable evidence even when finding that evidence requires expensive or heuristic search.

Concern	Reuse	New Forge work
Equality and linear arithmetic	Native grind state and solvers	Publish proved nonlinear consequences into that state
Theorem use	Existing annotations, E-matching, premise selection	Explicit open demands, conditional proof plans, targeted term construction
Ordered polynomials	Ring normalization and linear consequences	Exact quadratic, cone, and bounded-box certificates
Recursive definitions	Recursors, equation lemmas, induction tactics	Motive selection, dependency-safe generalization, invariant candidates
External reasoning	Existing ATP/SMT integrations	Uniform evidence contracts, residual-obligation handling, trust audit
Reproducibility	Existing proof scripts and tracing	One cross-engine receipt with scope and certificate dependencies

Table 1: The intended increment is coordinated proof construction, not replacement of established **Lean** infrastructure.

2 Related tools and the implementation position

2.1 A portfolio alone is not the proposal

Aesop already supplies best-first proof search with safe and unsafe rules, normalization, and tactic-based rule implementations. It is an appropriate host or library for the obligation-level AND/OR search described below.⁹ LeanHammer already combines premise selection and multiple proof-search or reconstruction routes, including several of the tools that would be useful in **Forge**. A wrapper that simply tries grind, Aesop, Duper, and an SMT solver would repeat this established approach.¹⁰

The architectural distinction is *cooperation through scoped mathematical obligations*. For example, the nonlinear engine does not merely attempt the final goal and fail. It can request a denominator-sign proof, prove a polynomial consequence under that sign, and hand the consequence back to the engine attempting a quantified lemma. Similarly, an induction branch can create algebraic obligations that reuse the same certificate search and replay machinery.

This is a design distinction, not a claim of historical priority for cooperative theorem proving. Failure-triggered interventions in grind have also been studied. The proposed baseline-preserving mode should be evaluated against that work rather than presented as a novel observation that one can try additional search after a failure.¹¹

⁹The Aesop project, implementation and user documentation. Full reference: [9].

¹⁰The LeanHammer project, current architecture and backend list. Full reference: [10].

¹¹Evan Wang et al., *Learned Interventions in Lean 4 grind*, 2026. Full reference: [11].

2.2 Concrete dependencies

Library or component	Intended role	Boundary or qualification
Lean core and Lean.Meta.Grind	Congruence closure, native arithmetic, typed expression processing, recursor applications	Pin the compiler version; hide unstable internal details behind one adapter
Aesop	AND/OR obligation search, rule scheduling, normalization	Use a controlled rule set; do not mark speculative invariant introduction as a safe rule
Mathlib tactics	ring, norm_num, positivity, linarith/nlinarith, cast lemmas	Treat their ordinary proof output as replay material; retain explicit side conditions
Duper and Lean-auto	First-order/equational proof search and translation infrastructure	Admit only reconstructed proofs in the original types and environment
Lean-SMT and cvc5	Optional quantified arithmetic or SMT search	Unsatisfiable answers with reconstruction holes remain conditional, not proved
SciPy HiGHS interface	Numerical finite-cone coefficient search in the prototype	Proposes a support and approximate weights; no numerical residual is accepted as an identity
SymPy	Exact rational support repair and test cross-checks	Not used by the independent certificate-replay command
New Forge modules	Reification, exact certificates, motive analysis, scope checking, receipt extraction	The principal implementation work; not provided by a one-line portfolio

Mathlib’s linear-arithmetic implementation already separates certificate search from proof construction, and its `linear_combination` infrastructure supports useful algebraic proof reconstruction. These are practical starting points rather than reasons to import a numerical optimizer into the trusted boundary.^{12,13}

Duper and Lean-auto are suitable sources of native proof-search and translation machinery.^{14,15} Lean-SMT explicitly warns that some reconstructed proofs can leave holes as goals. Its adapter must propagate those obligations instead of converting a backend status into success.¹⁶

Equality saturation can inform bounded representation search. However, the Lean-egg repository currently recommends `grind` and is deprecated; it should not be made a mandatory maintained dependency of a new tactic.¹⁷ The general e-graph literature remains useful for rebuilding and equality-class analysis, but a foreign untyped e-graph cannot replace Lean’s dependent typing discipline.¹⁸

¹²Mathlib, *Linearith frontend*. Full reference: [12].

¹³Mathlib, *LinearCombination*. Full reference: [13].

¹⁴The Duper project. Full reference: [14].

¹⁵The Lean-auto project. Full reference: [15].

¹⁶The Lean-SMT project, supported theories and proof reconstruction. Full reference: [16].

¹⁷The Lean-egg repository, deprecation notice. Full reference: [17].

¹⁸Max Willsey et al., *egg: Fast and Extensible Equality Saturation*. Full reference: [18].

3 User-facing behavior and success semantics

3.1 A small surface language

The intended interface is deliberately compact. The following syntax is a *proposal*, not syntax implemented by the supplied package.

```
by
  forge

by
  forge only [my_lemma, recurrence]

by
  forge (induction := [xs]) (generalizing := [acc])

by
  forge (engines := [grind, quadratic, cone, bernstein])

by
  forge? -- propose a deterministic replay script
```

An only mode must restrict the available theorem supply, not merely suppress some displayed names. Domain declarations, such as a certified box for variables, should be usable as local hypotheses rather than an undocumented configuration assumption. An explicit policy should control external processes and native-evaluation trust.

`forge?` should produce a proof script or certificate reference that is independent of later heuristic rankings. A user should be able to replace an exploratory call with a stable invocation and inspect the exact assumptions and lemmas used. The script should show an induction motive or a synthesized witness when either mattered.

3.2 Outcomes that do not collapse into a Boolean

An engine can return a candidate proof, a conditional constructor with premises, or useful progress. It can also return unsupported input, exhausted budget, cancelled work, stale context, or rejected evidence. A model or counterexample is a separate diagnostic object. It counts as a proof of a negation only after its interpretation in the original goal has been checked.

The outer tactic closes a goal only when the acceptance layer has an actual term of that goal's type, with no unassigned proof holes and no disallowed axiom dependencies. A conditional result is not a nearly proved theorem: it is a new AND node whose children all still need proofs. An unsupported real-algebraic expression is not silently replaced by a polynomial approximation.

For a failed search, a useful report might read:

```
Goal remains open.
Exact quadratic path: unsupported degree 4.
Cone path: no certificate in the current 36-atom dictionary.
Bernstein path: needs lower and upper bounds for y.
Requested obligation:  $y \leq 1$ .
No counterexample and no proof have been established.
```

These messages are intended behavior. The supplied prototypes use narrower status vocabularies and do not implement this complete user interface.

4 Scoped facts and proof obligations

4.1 Two graphs, not one bag of expressions

A *fact graph* stores propositions that have already been proved in a particular scope. An *obligation graph* stores propositions that a strategy would like to prove. Confusing these two graphs is the most serious architectural error to avoid.

A fact node contains an elaborated proposition, its proof, its local dependencies, the environment revision, and its semantic carrier information. A demand node contains the expected proposition, a goal metavariable, the relevant local context, candidate constructors, and a resource record. A hyperedge

$$(D_1, \dots, D_k) \longrightarrow D$$

represents an actual constructor of a proof of D from proofs of *all* the D_i . Alternative hyperedges are OR choices. Individual children of a hyperedge are AND obligations.

For instance, using a theorem

$$\forall t : \mathbb{R}, \quad 0 \leq t \rightarrow P(t)$$

to prove $P(q(x, y))$ creates the demand $0 \leq q(x, y)$. The theorem instance itself is valid immediately, but $P(q(x, y))$ is not a fact until the nonnegativity demand is discharged. This distinction supports backward reasoning without corrupting the saturation state.

4.2 Scope is part of the proposition identity

A safe cache key must include the elaborated term and enough context to interpret it. At a minimum, it records the environment revision, local declarations and their dependency order, universe parameters, and resolved typeclass instances. An expression hash is only an index hint. Reusing a proof still requires checking it against the current expected type.

A branch-local result proved using $x \neq 0$ cannot be published as an unconditional fact in a sibling branch. It can be retained either in the branch or, after abstraction, as a checked implication $x \neq 0 \rightarrow Q$. Likewise, a polynomial over a field of characteristic zero is not interchangeable with the same printed expression over a finite field. The instance arguments that select addition, order, division, and casts are semantically relevant.

To share across contexts, compute the proof’s actual free-variable dependency closure and abstract it to a theorem. Reinstantiate that theorem in the new context, then type-check the application. Do not persist raw local-variable or metavariable identifiers across elaboration sessions. A process-local state cache and a portable proof cache are different products.

4.3 Typed reification

The arithmetic adapter should return a reified polynomial *and a proof of its interpretation*. With ρ an environment of variable values, a suitable contract is

$$\text{reify}(e) = (p, \rho, h), \quad h : e = \text{eval}(p, \rho).$$

Variable atoms may themselves be compound **Lean** terms. Thus a polynomial in $\sin x$ and $\cos x$ can be handled algebraically by treating those terms as atoms, but identities connecting them require separately proved lemmas. The reifier must not assume algebraic independence to prove a false implication; it merely proves an identity valid for every assignment to the atoms.

The initial supported carrier should be $\mathbb{R}$, followed by a carefully stated ordered-field abstraction. Natural subtraction, integer division, truncated operations, modular arithmetic, and characteristic-dependent coefficients need separate adapters. For example, $a - b$ over $\mathbb{N}$ is not a ring subtraction, and division by a variable cannot be eliminated without a nonzero or sign obligation.

Unknown subterms are not automatically a failure. They may be reified as opaque variables when every inference remains valid for arbitrary values. Unknown *operations*, however, cannot be assigned ring semantics merely because their printed token is `+` or `*`.

5 The acceptance boundary and trust policy

5.1 The ordinary proof-term route

The preferred initial implementation has a small, conservative acceptance path. A worker submits a candidate constructor or serialized certificate. A host-side adapter reconstructs a term in the exact original context. It verifies the expected type, resolves all metavariables, records transitive dependencies, and lets `Lean` check the resulting declaration. Search can be incomplete, nondeterministic, numerically unstable, or even malicious without making a rejected proof acceptable.

An `Expr` field is not a proof-validation boundary. Calling a structure `CheckedProof` does not make its contents checked. In particular, reported dependency lists are diagnostics only; the acceptance layer must recompute dependencies from the accepted term. The supplied `DesignAPI.lean` deliberately calls submitted terms `Candidate` for this reason.

The default axiom policy should allow only the project’s explicitly approved assumptions, commonly the standard classical axioms used in ordinary Mathlib development. It must reject `sorryAx`, placeholder goals, and solver-specific assertions that are outside that policy. Custom project axioms may be allowed, but must be listed rather than hidden behind an unqualified claim of foundational trust.

5.2 Native evaluation is a real policy choice

There is an important distinction between a formally justified checker and the mechanism used to evaluate it. The current tactic reference says that `bv_decide` trusts the code generator and introduces axioms asserting its result; caching the LRAT file with `bv_check` does not itself change that evaluation route. The same reference distinguishes the proof-producing `cbv/decide_cbv` path from `native_decide`.¹⁹

Consequently, `Forge` should provide two clearly named policies. In the strict policy, certificate acceptance uses ordinary proof reconstruction or a proved checker evaluated through a proof-producing or kernel-reduction route. In an explicitly enabled accelerated policy, native evaluation may be used and its additional trust is recorded. Neither policy should be described by a vague statement that “the certificate was checked in Lean” without identifying the evaluation mechanism.

A strict SAT adapter is a substantial task: the Boolean abstraction, bit-vector encoding, clause semantics, LRAT checking theorem, and final transfer all need a complete checked path. Using stock `bv_decide` as an accelerated backend is useful, but it is not a free implementation of the proposed strict adapter. The certificate policy should expose this distinction to the user.

¹⁹Lean FRO, *Tactic Reference*, SAT integration and call-by-value evaluation. Full reference: [19].

5.3 Soundness of cooperation

Proposition 5.1 (Compositional acceptance). *Suppose every accepted fact in scope Γ is accompanied by a **Lean** term of its proposition; every accepted obligation edge is a term constructing its parent from all its children; and scope transfer is implemented by checked abstraction and application. Then any fully discharged obligation graph yields a term of its root proposition in its original scope.*

Proof. Topologically compose the constructor terms with the accepted child proofs. For a shared subproof, reuse the same checked term or a checked local declaration. Scope changes insert the stipulated abstraction and application. No open demand contributes a fact. The resulting root term is checked in the original context. The search policy does not enter this argument. $\square$

The graph used for ordinary composition must be acyclic. An apparent proof cycle can be accepted only when it is represented by a separately valid induction or well-founded recursion construction; merely finding a recurring goal is not a sound cyclic-proof rule.

6 Nonlinear ordered algebra: exact certificates

6.1 A common certificate language

Let $g_1, \dots, g_m$ be polynomials already known to be nonnegative, and let $h_1, \dots, h_r$ be polynomials already known to vanish. A general ordered-algebra certificate has the form

$$p = \sum_{j=1}^s c_j s_j^2 \prod_{i \in I_j} g_i + \sum_{k=1}^r t_k h_k, \quad c_j \in \mathbb{Q}_{\geq 0}. \quad (1)$$

Products can repeat hypotheses; the empty product is one. The t_k are arbitrary polynomials because their contribution vanishes. The prototype implements the first sum only. The equality-ideal part is a specified extension, not a tested feature.

Proposition 6.1 (Cone certificate soundness). *If identity (1) holds over an ordered field, all g_i evaluate nonnegatively, and all h_k evaluate to zero, then p evaluates nonnegatively.*

Proof. Each square is nonnegative. A finite product of nonnegative values is nonnegative, and multiplication by $c_j \geq 0$ preserves that property. Every $t_k h_k$ is zero. Sum the inequalities and rewrite using the exact polynomial identity. $\square$

This yields a direct **Lean** reconstruction path using square nonnegativity, multiplication and addition of nonnegative terms, rational arithmetic, and a polynomial identity proof. The expensive search for the certificate does not need to be trusted. Foundational checking of numerically discovered sum-of-squares evidence has a well-established precedent in Harrison’s work.²⁰

²⁰John Harrison, *Verifying nonlinear real formulas via sums of squares*, 2007. Full reference: [20].

6.2 Exact quadratic decomposition before numerical search

Quadratic goals deserve a dedicated exact path. For $p \in \mathbb{Q}[x_1, \dots, x_n]$ of total degree at most two, construct the unique symmetric matrix H with

$$p(x) = z^\top H z, \quad z = (1, x_1, \dots, x_n)^\top.$$

Put a linear coefficient $b_i x_i$ in entries $H_{0i} = H_{i0} = b_i/2$, and put an off-diagonal quadratic coefficient $a_{ij} x_i x_j$ in entries $H_{ij} = H_{ji} = a_{ij}/2$. Constant and square coefficients occupy the corresponding diagonal entries.

Use exact symmetric pivoting. If $H_{kk} = d > 0$, set v to the k th column and replace

$$H \leftarrow H - \frac{v v^\top}{d}. \quad (2)$$

Emit the nonnegative term $(v^\top z)^2/d$. The new k th row and column are zero. Continue until the residual matrix is zero. If a negative diagonal appears, or all diagonals are zero while an off-diagonal remains, this API returns no nonnegativity certificate.

Theorem 6.2 (Quadratic completeness over rational coefficients). *For a rational polynomial p of degree at most two that is nonnegative on all of $\mathbb{R}^n$, the exact pivot algorithm terminates with a certificate of the first form in (1), using at most $n + 1$ square terms.*

Proof. The homogenized form $z^\top H z$ is nonnegative when $z_0 \neq 0$, since it equals $z_0^2 p(z_1/z_0, \dots, z_n/z_0)$. By continuity it is also nonnegative at $z_0 = 0$. Thus H is positive semidefinite. A nonzero positive semidefinite matrix has a positive diagonal entry: a zero diagonal forces the corresponding row and column to vanish, as follows from its two-coordinate quadratic forms. A positive pivot leaves a positive semidefinite Schur complement on the remaining coordinates. Equation (2) is an exact identity and eliminates a coordinate. Induction on matrix size produces the claimed decomposition, with rational coefficients throughout. $\square$

The algorithm uses $O((n + 1)^3)$ rational arithmetic operations in a straightforward implementation. Bit complexity also depends on coefficient growth; “cubic” is not a claim of constant-cost arbitrary-precision arithmetic. A fraction-free variant and sparsity-aware pivot order are worthwhile later optimizations.

This mechanism automatically finds witnesses for $(x + 3y)^2$ and $(x - \frac{1}{2})^2$ after expansion. These examples matter because a dictionary containing only squares of monomials and pairwise sums or differences need not contain either required square. The accompanying `quadratic.py` implements the exact pivot path and replays its answer with the general cone checker.

6.3 Finite-dictionary cone search

For higher-degree goals, start with a transparent bounded search. Select polynomials s from monomials and small linear combinations of monomials, multiply their squares by bounded products of the available g_i , and discard terms above a degree cap. Each surviving expanded polynomial a_j is a column in a coefficient matrix A . If b is the coefficient vector of p , solve

$$Aw = b, \quad w \geq 0. \quad (3)$$

The prototype uses SciPy’s `linprog` with the HiGHS method. It uses an all-ones objective simply to obtain a nonnegative representation; it does not implement an optimizer for `Lean` proof size.²¹

²¹SciPy documentation, `scipy.optimize.linprog`. Full reference: [21].

The floating-point solution proposes a support. Approximate weights are converted to rationals with bounded denominators. The exact checker expands every term and compares every rational coefficient. If reconstruction fails, the prototype solves the proposed support’s linear equations by exact rational Gaussian elimination through SymPy, then checks the answer again. Setting free parameters to zero is a heuristic repair policy and can miss a feasible nonnegative solution.

```

cone_search(target, proved_nonnegative_hypotheses, degree):
    atoms := bounded_square_products(hypotheses, degree)
    A, b := exact_coefficient_matrix(atoms, target)
    candidate := numerical_nonnegative_solution(A, b)
    if no candidate: return UNKNOWN
    weights := reconstruct_rationals(candidate)
    if exact_replay(target, atoms, weights): return CERTIFICATE
    weights := exact_support_repair(A, b, candidate.support)
    if exact_replay(target, atoms, weights): return CERTIFICATE
    return RECONSTRUCTION_REJECTED

```

An infeasible result for (3) only says that the current dictionary did not yield a certificate. It does not refute the polynomial inequality. Even a mathematically exact LP infeasibility certificate would refute *that coefficient representation problem*, not necessarily the original theorem. A numerical dual vector is therefore a search hint, not an automatic counterexample to $p \geq 0$.

6.4 An example of hypothesis-sensitive search

Suppose $0 \leq x \leq 1$ and $0 \leq y \leq 1$. The expression

$$p = x(1 - x)y(1 - y)$$

is nonnegative because it is a product of four known nonnegative terms. A solver searching only global sums of squares cannot prove it globally: the polynomial is not globally nonnegative. A certificate with $s = 1$ and $I = \{x, 1 - x, y, 1 - y\}$ proves the local goal immediately.

The prototype deliberately includes this example with a hypothesis-product cap of four; most other cone experiments use a cap of two. The degree cap alone is not a substitute for the hypothesis-product cap. The implementation also records each hypothesis reference, so replay with a missing or out-of-scope assumption fails.

For

$$p = (x - y)^2 + (xy - 1)^2,$$

the search finds two square terms with no hypotheses. The role of the tactic is to discover this representation from the expanded polynomial and construct the proof, not to assume that the two squares were supplied as hints.

6.5 From the bounded dictionary to richer SOS search

A production extension can replace a hand-selected square dictionary with a Gram-matrix search. For a monomial vector $m_d(x)$, seek

$$p = m_d(x)^T Q m_d(x), \quad Q \succeq 0,$$

or use several such matrices multiplied by selected hypothesis products. Coefficient matching is linear in the matrix entries. A numerical semidefinite solver proposes matrices; acceptance requires exact rational coefficient identities and an exact positive-semidefinite decomposition. Eigenvalues that are merely approximately nonnegative are not evidence.

The singular case is the difficult one. Rational rounding of an almost singular matrix can destroy positivity or coefficient matching. Reconstruct a rational affine solution space to the coefficient constraints, identify a candidate face or kernel, and solve the remaining exact constraints. When this fails, return unknown or increase the basis. This SDP extension is *not implemented in the package*; the exact quadratic solver only handles the special case where the polynomial determines its augmented Gram matrix uniquely.

Equality constraints can be handled by extending (3) with unrestricted coefficients for bounded multiples of h_k , or by reducing p and each cone atom modulo an equality ideal while retaining reconstruction witnesses. The latter can reuse ring-side infrastructure, but ideal membership alone does not prove positivity. Every reduction needs its explicit $\sum t_k h_k$ identity.

6.6 Strict inequalities, casts, and denominators

A direct nonnegativity certificate proves $p \geq 0$, not $p > 0$. A simple strict certificate proves $p - \varepsilon \geq 0$ for a rational $\varepsilon > 0$. Alternatively, a contradiction certificate may combine $p \leq 0$ with strictly positive factors. These two routes should have distinct checker rules so a nonnegative quantity is never silently promoted to a positive one.

For a rational expression a/b , a demand about b must precede denominator clearing. Multiplying an inequality by b needs its sign; multiplying by b^2 still requires care when $b = 0$ and must not change the original expression’s semantics. A branch split $b = 0 \vee b \neq 0$ can be useful, but the zero branch must be proved under the actual field-division convention.

A theorem over $\mathbb{Z}$ may sometimes be proved after embedding into $\mathbb{R}$, but the cast transport and assumptions need proofs. Real nonnegativity does not decide integer-only divisibility or rounding phenomena. Conversely, an inequality valid for integer points may fail over real points, so the real relaxation’s failure says nothing decisive about the original integer goal.

7 Bernstein certificates on bounded domains

7.1 Why a second nonlinear representation helps

A global sum-of-squares search and a bounded-domain positivity proof solve different problems. A polynomial can be easy to bound on a box but awkward to represent in a small global cone. Bernstein coefficients give a particularly simple exact certificate: a polynomial is a convex combination of its coefficients on the unit box.

The formal use of Bernstein bounds is established; NASA’s PVS library includes a formalized Bernstein approach for polynomial inequalities. The proposal here is a compatible certificate service inside the **Forge** obligation architecture, not a new mathematical positivity theorem.²²

Let

$$B = \prod_{i=1}^n [\ell_i, u_i], \quad \ell_i, u_i \in \mathbb{Q}, \quad \ell_i < u_i,$$

²²NASA Langley Formal Methods, PVS Bernstein library. Full reference: [22].

and substitute $x_i = \ell_i + (u_i - \ell_i)t_i$. Write the resulting polynomial as

$$\tilde{p}(t) = \sum_{\alpha \leq d} a_\alpha t^\alpha.$$

The componentwise degree bound is $d = (d_1, \dots, d_n)$. The tensor Bernstein basis is

$$B_\beta^d(t) = \prod_{i=1}^n \binom{d_i}{\beta_i} t_i^{\beta_i} (1 - t_i)^{d_i - \beta_i}, \quad 0 \leq \beta \leq d.$$

The corresponding coefficients are

$$b_\beta = \sum_{\alpha \leq \beta} a_\alpha \prod_{i=1}^n \frac{\binom{\beta_i}{\alpha_i}}{\binom{d_i}{\alpha_i}}. \quad (4)$$

Only indices $\alpha_i \leq d_i$ occur, so the denominators are positive integers. For degree zero in a coordinate, the associated factor is one.

7.2 The certificate theorem and its proof

Lemma 7.1 (Bernstein bounds). *If $\tilde{p} = \sum_{\beta \leq d} b_\beta B_\beta^d$, then, on $[0, 1]^n$,*

$$\min_{\beta} b_\beta \leq \tilde{p}(t) \leq \max_{\beta} b_\beta.$$

In particular, nonnegative coefficients prove nonnegativity, and strictly positive coefficients prove strict positivity.

Proof. Each basis term is nonnegative on the unit box. The binomial theorem in each coordinate gives

$$\sum_{\beta \leq d} B_\beta^d(t) = \prod_i (t_i + (1 - t_i))^{d_i} = 1.$$

Thus the expansion is a convex combination of the coefficients. Bounding every coefficient below and above proves the inequalities. $\square$

The checker need not trust formula (4). It can expand the supplied basis expression exactly and compare it to the affine-substituted polynomial. The prototype deliberately uses this opposite conversion for replay. The producer's coefficient-conversion formula and the verifier's identity computation are therefore different code paths, although they still share the sparse polynomial arithmetic and neither is formally verified.

7.3 Subdivision with a coverage certificate

When some coefficients are negative, split one coordinate interval at an interior rational point and certify both children. A certificate is a binary tree. A leaf stores a degree vector and rational coefficients. An internal node stores an axis and a cut. The checker derives both child boxes from the parent; the producer is not allowed to supply unrelated boxes and claim that they cover the domain.

The prototype chooses a longest relative-width coordinate and bisects at its rational midpoint. Relative width avoids privileging a coordinate merely because its units make its raw interval much larger. Deterministic tie-breaking makes the receipt reproducible.

```

certify_box(p, box, depth):
  coefficients := exact_bernstein_coefficients(p, box)
  if all coefficients satisfy the required sign:
    return Leaf(degrees, coefficients)
  if resource limit reached: return UNKNOWN
  axis := longest_relative_width(box)
  cut := exact_midpoint(box[axis])
  left, right := split(box, axis, cut)
  L := certify_box(p, left, depth + 1)
  R := certify_box(p, right, depth + 1)
  if either child is UNKNOWN: return UNKNOWN
  return Split(axis, cut, L, R)

```

A final checker validates every cut, every coefficient sign, every leaf identity, and the whole tree. The tree is essential: proving nonnegativity on a collection of sampled sub-boxes is insufficient without proving that they cover the original box.

Proposition 7.2 (Subdivision soundness). *An accepted Bernstein tree proves its claimed sign throughout the original box.*

Proof. At a leaf, use the exact identity and the Bernstein bounds lemma. At a split, the two closed child intervals cover the parent interval and all other coordinates are unchanged. Combine the child implications by cases on the split coordinate. Induction on the tree proves the root claim. $\square$

7.4 A qualified completeness result

Proposition 7.3 (Strictly positive compact-box case). *Let p be a fixed real polynomial that is strictly positive on a compact rational box of positive widths. Under a fair sequence of subdivisions whose maximum normalized cell diameter tends to zero, sufficiently fine cells all have strictly positive Bernstein coefficients at the polynomial's fixed degree. Hence the unbounded subdivision procedure eventually obtains a finite positivity certificate.*

Proof. Compactness gives a minimum $\mu > 0$. For a cell with lower corner a and widths w , write the exact Taylor expansion of $p(a + w \odot t)$. Its constant term is $p(a)$. All other power coefficients contain at least one factor from w ; because the degree is fixed and derivatives are bounded on the box, their absolute sum tends uniformly to zero as the maximum width tends to zero. In (4), each binomial ratio lies in $[0, 1]$. Therefore every Bernstein coefficient differs from $p(a)$ by at most a uniform quantity tending to zero. Eventually this quantity is less than $\mu/2$, so every coefficient is positive. A sufficiently fine finite subdivision supplies the tree. $\square$

This is not a completeness result for all nonnegative polynomials. Consider $p(x) = (x - \frac{1}{3})^2$ on $[0, 1]$. The dyadic cell containing $1/3$ always has the root in its interior. If its endpoints are $\ell < u$, the middle quadratic Bernstein coefficient is

$$(\ell - \frac{1}{3})(u - \frac{1}{3}) < 0.$$

No finite midpoint-only subdivision can certify that cell by nonnegative degree-two coefficients. The exact quadratic engine proves the same inequality immediately. This is a concrete reason to cooperate across representations instead of escalating one representation's resource cap indefinitely.

The tensor coefficient count is $\prod_i (d_i + 1)$ per leaf. Both dimension and the number of cells can dominate runtime. For larger problems, exploit variable sparsity, split only relevant coordinates, cache affine substitutions, and route low-rank or separable pieces back to cone reasoning. None of these optimizations changes the leaf acceptance theorem.

8 Goal-directed theorem use and demand slicing

8.1 A need is not an E-matching fact

Suppose a theorem concludes $R(t)$ if $A(t)$ and $B(t)$ hold. Ordinary matching can discover the theorem instance. A goal-directed planner can additionally decide that proving $A(t)$ and $B(t)$ is a promising way to obtain a currently needed $R(t)$. The planner should retain this as a conditional constructor, not add $R(t)$ to the fact store.

Forge should maintain two indices. A theorem-conclusion index maps elaborated head symbols and result types to candidate backward rules. A premise/trigger index maps already proved facts or newly created terms to potential forward consequences. Typeclass arguments and universe constraints remain part of elaboration; a textual symbol match only proposes a candidate.

A demand carries an estimated cost, a depth, and a relevance ancestry. Candidate lemmas are first filtered by type compatibility, then ranked by overlap with the demand’s symbols and proved facts. Useful deterministic features include the number of immediately discharged premises, the number of new variables, expected branch count, and previously measured replay cost. A learned ranker may order candidates, but it cannot authorize an inference.

Existing `grind` triggers and premise selection should remain available. The new contribution is the explicit lifecycle of an unproved premise, including calls to induction, nonlinear certificates, or witness construction. A backchaining rule that only reimplements an existing directional trigger is not an improvement worth maintaining.

8.2 A finite fragment with a complete slicing argument

The prototype isolates a simple relevance mechanism on *ground* Horn rules

$$p_1 \wedge \cdots \wedge p_k \rightarrow q.$$

It builds a reverse index by conclusion, starts from the requested target, and recursively includes every rule whose conclusion is needed, together with its premise demands. Forward chaining then runs only on the selected rules.

Proposition 8.1 (Demand slicing preserves ground-Horn derivability). *For a finite set of ground Horn rules and initial facts, the target is in the full least fixed point if and only if it is derivable from the rules selected by the complete backward-demand slice.*

Proof. The selected rules are a subset of the full rule set, so sliced derivability implies full derivability. Conversely, consider a finite derivation of the target in the full system. Every rule concluding the target is selected. Recursively, every rule concluding a premise used in the derivation is selected. Induction on derivation height therefore reproduces the derivation in the slice. Cycles without a fact-supported derivation add no facts and do not invalidate the argument. $\square$

The word *complete* matters: selecting only the highest-ranked incoming rule can lose derivability. In the *Lean* setting, aggressive relevance pruning must therefore coexist with a fair fallback or be explicitly described as a bounded heuristic.

The producer uses premise counters and an index from each fact to rules waiting for it. Each newly known fact decrements the appropriate counters; a rule fires when all distinct premises are known. The receipt retains only the applications needed for the target, and an independent replay checks their topological order and premise availability.

With R rules and L total premise occurrences, building the indices costs $O(R + L)$. The selected closure then costs $O(R_s + L_s)$, ignoring symbol-storage costs. An implementation that rebuilds the full index for a single query still pays the initial $O(R + L)$ cost. The accompanying experiment reports index work separately; it does not claim that a reduction in rule firings translates directly into the same factor of total speedup.

8.3 From the toy fragment to typed quantified search

A production implementation must extend this idea carefully. E-matching works modulo known equalities, so demands should carry a stable expression and subscribe to equality-class changes, not cache a representative forever. Multi-premise rules require indexed joins over substitutions, with semi-naive scheduling so a new fact only recombines the joins it can change. Deduplicate by theorem identifier, instantiated type parameters, and canonicalized substitution under the current equality state.

The demand planner can also synthesize a term that has not yet occurred. Restrict the initial grammar to local variables, relevant constants, constructors, and a bounded set of type-correct applications. For affine arithmetic witnesses, use a more specialized symbolic solver rather than enumerating arbitrary syntax. Elaborate every candidate in a saved context; an unsuccessful unification attempt must not leave metavariable assignments that affect another candidate.

When saturation needs genuinely different equational inference, use a native prover such as Duper or a reconstructing translation pipeline rather than extending the toy Horn interpreter into an ad hoc higher-order prover. The exported facts still pass through the same acceptance layer. The ground-Horn experiments establish the slicing mechanism only; they do not measure E-matching, quantified theorem retrieval, or grind performance.

9 Induction, generalization, and invariant construction

9.1 Why local saturation is not enough

A recursive program specification often cannot be established by repeatedly unfolding at an unknown input. Unfolding a list function at an arbitrary list exposes a case split; proving the recursive case requires an induction hypothesis, and a tail-recursive implementation may require that hypothesis at a different accumulator.

The proposed planner should recognize recursion-bearing goals and consider recursor-based proof plans. *Lean* already provides structural and function-oriented induction tactics; the problem is choosing and instantiating the appropriate plan, not inventing another trusted induction rule.[19] The prototype implements only a small first-order fragment with lists, natural numbers, defining equations, congruence rewriting, and structural induction.

9.2 The accumulator example

Define reverse with an accumulator by

$$\text{ra}([], a) = a, \quad \text{ra}(x :: xs, a) = \text{ra}(xs, x :: a).$$

The desired theorem is

$$\text{ra}(xs, a) = \text{rev}(xs) ++ a. \tag{5}$$

Inducting on xs while keeping a fixed gives an induction hypothesis only at a . The recursive call in the step has accumulator $x :: a$, so that hypothesis does not match. Increasing E-matching effort cannot turn a theorem about one fixed accumulator into a universally quantified theorem about all accumulators.

Instead use the motive

$$M(xs) := \forall a, \quad \text{ra}(xs, a) = \text{rev}(xs) ++ a.$$

In the step, instantiate the induction hypothesis at $x :: a$:

$$\begin{aligned} \text{ra}(x :: xs, a) &= \text{ra}(xs, x :: a) \\ &= \text{rev}(xs) ++ (x :: a) \\ &= (\text{rev}(xs) ++ [x]) ++ a \\ &= \text{rev}(x :: xs) ++ a. \end{aligned}$$

The third line uses associativity and the defining equations of `append`. This is *local motive generalization*: it strengthens an intermediate induction statement and then specializes it. It does not silently change the user's theorem or remove its assumptions.

9.3 A concrete generalization algorithm

Read the defining equations or generated recursion information for the recursive functions appearing in the goal. For each candidate induction argument, compare each recursive call's nonrecursive arguments with the corresponding arguments in the defining equation's left side. If an argument changes, mark the free local variables in the actual supplied argument as generalization candidates.

For (5), the comparison is a versus $x :: a$, so the candidate set contains the accumulator. For a length accumulator with recurrence

$$\text{la}(x :: xs, n) = \text{la}(xs, n + 1),$$

the comparison is n versus $n + 1$, so the numeric accumulator is marked. Unchanged parameters need not automatically be generalized.

In **Lean**, extend the marked set by dependency closure: if a local declaration's type or value depends on a marked variable, it may need to be reverted too. Revert in a dependency-safe order, construct the motive, and reintroduce parameters in each branch. A dependent parameter cannot be generalized independently of the indices that make its type meaningful.

Rank candidate induction plans by recursive-call alignment, size of the generalized context, and expected branch count. Try the cheapest plan, retain alternatives under a bounded search, and prefer the generated function induction principle when it better matches a well-founded or nested recursive definition. Do not force every function into a structural induction on its first printed argument.

The supplied Python analysis has deliberately narrower scope: first-argument structural recursion for list functions and no dependent types. The induction subject is supplied in its tests. Automatic subject selection, dependent-context closure, mutual induction, and function-induction integration remain specified work, not tested capabilities.

9.4 Replay must restrict the induction hypothesis

An induction receipt contains the subject, generalized parameters, and equality proof traces for base and step. The checker reconstructs the two branch goals itself. In the step, the induction hypothesis is available at the *immediate tail or predecessor*; only generalized parameters are pattern variables that can vary.

Allowing the induction hypothesis to rewrite at the original input would make it a circular proof of the goal. Allowing an un-generalized accumulator to vary would strengthen the induction hypothesis without justification. The prototype explicitly rejects both kinds of overreach in its representation: the predecessor is a fixed eigenvariable, and only named generalized parameters are instantiated.

Accepted lemmas are installed for subsequent proofs only after their induction receipts replay. The eight-example sequence uses append and addition lemmas as prerequisites for later reverse and accumulator proofs. Its statistics therefore describe a small dependency-ordered development, not eight unrelated goals with a hidden unlimited oracle.

9.5 Invariant synthesis beyond generalization

Sometimes generalizing existing variables is insufficient because the needed relation is absent. The next stage is constrained invariant synthesis. Extract candidate expressions from the function’s inputs, recursive-call arguments, and postcondition; anti-unify recurring shapes; and enumerate a bounded grammar of equalities, inequalities, conjunctions, and accumulator relations. For arithmetic invariants, solve template coefficients symbolically when possible.

A candidate invariant is a new theorem to prove. Test it on small concrete inputs to discard bad candidates cheaply, but never accept it from testing. Generate its base and step obligations and discharge both through ordinary proof construction. Only then install it as a lemma and use it for the original goal.

For example, an accumulator invariant can have the shape

$$\text{foldAcc}(xs, a) = F(xs) \star a$$

where $\star$ is selected from operations actually present in the recurrence. The planner can test whether associativity and identity laws for $\star$ would suffice, creating those laws as explicit demands when they are not already available. For indexed inductive types, candidate templates must preserve all indices and coercions; syntactic anti-unification alone is not a proof of a well-typed invariant.

There is no claim of completeness for invariant synthesis. A useful initial acceptance milestone is a small suite of tail-recursive list, tree, and numeric functions where the synthesized invariant is proved without user-provided hidden strengthening lemmas. A future benchmark must report the invariant grammar and candidate count.

10 Witnesses, quantified reasoning, and finite search

10.1 Constructing rather than only refuting

For an existential goal, a direct witness can be far cheaper than unrestricted contradiction search. The planner should classify the goal by a supported witness grammar. Affine Presburger constraints suggest affine integer expressions and divisibility-aware offsets; inductive membership suggests constructors; an image or range goal suggests a preimage supplied by an existing equality.

For example, to prove

$$\forall x \in \mathbb{Z}, \quad \exists y \in \mathbb{Z}, \quad x < y \wedge 2 \mid (y - x),$$

a specialized affine witness search can propose $y = x + 2$. The acceptance path then constructs the existential witness, proves the inequality, and proves divisibility. Finding a witness at a few sampled integers is not enough: the symbolic expression must be verified for an arbitrary x .

For a grammar-based fallback, enumerate well-typed candidates by size, use counterexamples only to eliminate candidates, and submit survivors to the ordinary prover. A purported counterexample must satisfy the original assumptions to be decisive. A model of an abstraction may be useful feedback without being a counterexample to the original `Lean` goal.

10.2 Quantifier and equality adapters

Use premise selection to export a small, explicit theorem set to Duper, Lean-auto, or Lean-SMT. Track every transformation: universal instantiation, lambda lifting, monomorphization, Skolemization where applicable, and theory abstraction. The result must reconstruct in the original dependent types. A successful first-order proof under a silently stronger premise is not an acceptable proof of the original theorem.

For model-guided instantiation, a model can prioritize terms on which a quantified formula appears violated. Instantiating a universally quantified theorem at a well-typed term is always legitimate; the model does not justify the theorem itself. Store the actual theorem application, not a statement that the model said a particular instance was needed.

Do not claim a general complete combination of arbitrary `Lean` theories. Classical theory-combination completeness theorems require hypotheses that arbitrary typeclass-parametrized or finite structures need not satisfy. The proposed combination mechanism is operationally simple and sound: exchange checked consequences in the native logic, and accept incompleteness when no supported engine can produce a useful consequence.

10.3 Boolean and finite-domain reasoning

A Boolean/SAT adapter should exploit the existing specialized ecosystem rather than make the outer proof planner perform millions of explicit case splits. The challenge is the evidence path. For strict mode, prove the equivalence or implication between the original goal and its Boolean encoding, replay the satisfiability certificate with an approved evaluation route, and transfer the result back. In accelerated mode, explicitly document and audit the native trust described earlier.

Finite-domain enumeration can also be useful for small types, but cardinality assumptions must be real proofs, not guessed machine-word widths. Treating an unknown function as an unconstrained symbol may prove a stronger abstraction and is then safe; interpreting a countermodel of that abstraction as a concrete counterexample is not automatically safe.

These adapters are part of the proposed production scope, not implemented by the Python Horn prototype. That prototype contains neither a SAT solver nor an SMT translation.

11 Cooperative scheduling and integration with grind

11.1 A staged control loop

The recommended first implementation is sequential and deterministic. Run the stock grind path with its reserved budget. If it does not close the goal, classify the remaining structure and activate a small number of obligation-producing engines. Newly proved facts can be fed into a fresh or explicitly managed native grind state, while induction and witness plans operate above that state.

```
forge(goal, policy):
  saved := save_exact_elaboration_state(goal)
  result := run_stock_grind(saved, reserved_budget)
  if accepted(result): return replay_receipt(result)
  restore(saved)
  plan := make_root_obligation(goal)
  while budget remains:
    obligation, engine := choose_fairly(plan)
    reply := engine.run(snapshot(obligation), local_budget)
    match reply:
      candidate    => reconstruct_and_check_then_publish(reply)
      conditional  => add_all_premises_and_constructor(reply)
      progress     => validate_each_fact_then_reschedule(reply)
      other        => record_reason_without_asserting_facts(reply)
    if root_has_checked_proof(plan):
      return minimize_and_replay(plan)
  return explicit_open_obligations(plan)
```

This is not a claim that ordinary grind failure returns a reusable internal snapshot through a stable public API. The first adapter can restore the saved elaboration state and restart with newly proved lemmas. Incremental state reuse should be introduced only after the exact v4.34.0 interface has been wrapped and tested. The existing plugin interface and CPS actions provide a practical route for tighter integration.[8, 7]

11.2 Engines should subscribe to events

An inequality engine should not rerun after every unrelated equality. Track the dependencies of each reified problem: variable atoms, hypotheses, and domain bounds. Reschedule when one of those changes or when a demanded fact becomes available. Similarly, an induction plan should not restart because an unrelated propositional fact was added.

A simple deterministic score for an eligible engine is

$$\text{score}(E, D) = \frac{\text{estimated useful progress}(E, D)}{1 + \text{estimated search cost} + \text{estimated replay cost}} - \lambda \text{ new-branch penalty}.$$

The exact score is a tuning parameter, not a correctness condition. Include an aging term or round-robin quota so a plausible but low-ranked plan is not starved indefinitely. Resource counters must charge reification, theorem elaboration, certificate decoding, and replay, not just an external solver's reported solve time.

If concurrency is added later, isolate each worker's elaboration state. Do not let workers mutate the same metavariable assignments or native solver state. The host validates returned candidates against their source revision before committing them. A stale result may be reusable after re-elaboration, but must not be assigned directly to a new goal that merely has a similar printed form.

11.3 The precise non-regression guarantee

Let G_B be stock grind run with budget B in a fixed environment and deterministic configuration. If **Forge** first gives that exact computation its own unmodified budget, then every goal solved by G_B is also solved by that **Forge** mode, up to wrapper and validation overhead. Additional engines use additional resources.

This does *not* imply dominance at the same total budget. If the total allowance is B and **Forge** spends part of it on reification, retrieval, or induction, it can lose goals that stock grind would solve. Both policies are useful: a baseline-preserving mode with a larger total allowance, and a fixed-total-budget mode whose gains and regressions are measured honestly. Benchmark them separately.

Proof size creates another tradeoff. A very short numerical search may return a large exact certificate whose reconstruction is expensive. The scheduler should optimize accepted-proof cost, not just discovery latency. The receipt needs separate fields for search, elaboration, certificate replay, kernel validation, and final term size.

12 A worked cross-engine proof plan

12.1 An accumulator inequality

Define a list computation over real numbers by

$$\begin{aligned}\text{energyAcc}([], a) &= a, \\ \text{energyAcc}(x :: xs, a) &= \text{energyAcc}(xs, a + (x - 1)^2).\end{aligned}$$

The goal is

$$\forall xs\ a, \quad a \leq \text{energyAcc}(xs, a). \quad (6)$$

An outer induction plan identifies xs as structural and detects that the recursive accumulator changes. It generalizes a . The base is reflexive. In the step, the induction hypothesis gives

$$a + (x - 1)^2 \leq \text{energyAcc}(xs, a + (x - 1)^2).$$

The remaining demand is $a \leq a + (x - 1)^2$. The ordered-algebra engine proves $(x - 1)^2 \geq 0$, the linear engine adds a , and transitivity closes the step.

If the update were an expanded polynomial such as

$$q(x, y) = x^2 + 6xy + 9y^2,$$

the same plan could use exact quadratic decomposition to discover $q = (x + 3y)^2$. If the update were only nonnegative under a loop invariant, the invariant must be proved and threaded through the induction motive. The arithmetic certificate alone cannot supply an invariant about recursive calls.

The package contains an ordinary **Lean** proof of (6) as an uncompiled integration target. It is not claimed that the full planner generated that proof. Its purpose is to make the desired cross-engine constructor sequence concrete and easy to validate in the pinned project.

12.2 A quantified predicate with a nonlinear side condition

Assume a user theorem

$$H : \forall t : \mathbb{R}, \quad 0 \leq t \rightarrow P(t),$$

and request $P((x - y)^2 + (xy - 1)^2)$. The backward index finds H and creates the side demand. The cone engine reconstructs two squares, proves the demand, and the parent constructor applies H . The final receipt contains the theorem application and the nonnegativity certificate, not a log of every rejected candidate.

This example is deliberately elementary so the proof architecture is transparent. It is not evidence that current `grind` fails on this exact statement; existing annotations or supplied lemmas may already solve it. A genuine comparison must test the same imported environment and theorem supply. The architecture matters more on larger cases where side conditions lead to additional obligations and different engines.

13 Prototype implementation and measured results

13.1 What was executed

The experiments ran in Python 3.13.5 on Linux x86_64, with NumPy 2.3.5, SciPy 1.17.0, and SymPy 1.14.0. The random seed is 20260914. The primary output is `results/summary.json`; row-level measurements are in CSV files. The complete run passed 1,680 explicit assertions and took approximately 2.27 seconds in the recorded environment. That duration is a single-run mechanism-test measurement, not a calibrated performance claim.

Twenty-four certificate bundles were independently replayed by `prototype/replay.py`. That command imports no numerical optimizer or symbolic algebra package and uses only Python’s standard library. It still trusts the Python interpreter and the hand-written checker implementation. Exact rational arithmetic removes floating-point tolerance from acceptance, but it does not constitute formal verification of the checker.

The `Lean` check runner recorded `NOT_RUN`: no local `Lean/Lake` executable was available. The compiler-facing files were not compiled, and no theorem-success or timing comparison against `grind` was performed. The package contains a pinned example project and a runner so this remaining boundary can be tested without confusing it with the completed Python experiments.

13.2 Headline mechanism comparisons

Experiment	Recorded result	Interpretation
Ten curated cone goals	0 / 4 / 9 solved	Monomial-square / pair-square / hypothesis-product dictionaries; same selected cases
Exact quadratic path	4/4 curated; 60/60 generated	Rational PSD decomposition, including two dictionary limitations
Nine true box goals and one false control	3/9 to 8/9 true goals	No subdivision versus depth-12 adaptive Bernstein; false control never certified
Eight induction theorems	6/8 to 8/8	Generalization disabled versus enabled; subjects supplied
Twelve Horn workloads, 100-firing cap	3/12 to 12/12	Un sliced versus demand-sliced forward chaining
Random ground-Horn graphs	500/500 mode comparisons agree	250 graphs, two modes, compared with an independent least-fixed-point computation

Table 3: These are ablations of the supplied Python mechanisms, not measurements of `grind`, `Aesop`, or `LeanHammer`.

13.3 Cone and quadratic experiments

The ten curated cone examples include expanded squares, quartic sums, products requiring nonnegative hypotheses, and a rationally shifted square. They were selected to exercise the dictionaries, so the zero successes of the diagonal-only dictionary are not a claim about the prevalence of hard polynomials. Four succeed after adding pairwise monomial sums and differences, and nine succeed after allowing the selected products of hypotheses.

The remaining rationally shifted square is a useful failure, not a bug in the verifier. A separate positive rank-one example, $(x + 3y)^2$, also fails in the small default dictionary and succeeds when that square is explicitly added. The exact quadratic path then proves both examples automatically without numerical search, as well as two other curated cases and 60 generated rational PSD quadratics in one through five variables.

Thirty additional cone targets were constructed from three randomly selected dictionary atoms with positive rational weights. All were recovered and replayed. This is an implementation stress test, not an unbiased benchmark: the generation process intentionally guarantees a certificate in the tested dictionary. The random quadratic family has a similarly explicit construction bias.

The cone mutation checks reject an altered target differing by 10^{-20} , a negative weight, an out-of-range hypothesis reference, a floating-point weight, and a mismatched polynomial arity. The quadratic path rejects four indefinite or slightly negative controls. These tests help ensure that tolerance-based acceptance has not accidentally leaked into the exact checker; they do not prove the checker free of every defect.

13.4 Bernstein experiments

Polynomial on $[0, 1]^2$	Tree nodes	Leaves	Max. depth
$(x - y)^2 + 1/100$	43	22	6
$(x - 1/3)^2 + (y - 2/3)^2 + 1/20$	11	6	4
$(xy - 1/3)^2 + 1/25$	13	7	4
$(x^2 - y)^2 + 1/20$	19	10	4
$(x - 1/2)^2$	3	2	1

Table 4: Selected accepted exact subdivision certificates. Strict positivity was required for the first four; nonnegativity for the last.

The three already-certifiable root boxes are $1 + x^2$, $1 - x(1 - x)y(1 - y)$, and $x(1 - x)y(1 - y)$. Subdivision adds the five successes shown in the table. The ninth true goal, $(x - 1/3)^2 \geq 0$, remains unknown at depth twelve for the mathematical reason proved earlier. The deliberately false goal $-1 \geq 0$ also remains uncertified; the prototype is a certificate searcher, not a counterexample-reporting decision procedure.

Seven mutation checks reject incorrect coefficients, floating-point coefficients, a boundary cut, an invalid axis, reusing parent-box leaves as child-box certificates, an excessive degree declaration, and promotion of a zero certificate to strict positivity. The coverage and strictness checks are just as important as the polynomial arithmetic.

13.5 Induction experiments

The theorem sequence is append right identity, append associativity, addition right zero, addition right successor, reverse over append, reverse involution, reverse-accumulator correctness, and length-accumulator

correctness. Without generalization, the first six replay. With recurrence-directed parameter generalization, all eight replay. The changed parameters are exactly the list accumulator in reverse and the natural accumulator in length.

Eighteen induction-certificate mutation checks are rejected. They include generalizing the induction subject itself, referring to an unregistered rewrite rule, and removing an accumulator from the generalized set while retaining a proof that used the stronger hypothesis. A false reverse conjecture remains unproved in both modes.

The interpreter’s equational definitions and induction checker form a small, hand-written model of the intended reasoning. They are not a parser or elaborator for `Lean`, do not represent dependent types, and do not establish a verified embedding theorem into `Lean`. The complete proof traces are supplied to make the exact mechanism inspectable.

13.6 Demand-slicing experiments and the indexing cost

Each synthetic workload contains a relevant chain of length 8, 16, or 32 and 0, 100, 1,000, or 10,000 decoy rules. Decoys are deliberately ordered before the useful chain. With a 100-rule-firing limit, unsliced FIFO forward chaining solves the three cases without decoys; the demand-sliced mode solves all twelve.

For the chain of length 16 with 10,000 decoys, the sliced mode fires 16 rules while the unsliced mode spends its 100 allowed firings on irrelevant work. Both still inspect 10,016 rules to build the conclusion index. Thus the experiment demonstrates budget-sensitive relevance control, not a 625-fold end-to-end speedup. A prebuilt persistent index would change amortized costs, but that optimization is not implemented in the prototype.

On 250 additional random graphs, including empty-premise rules, duplicate premises, cycles, and unreachable targets, both modes agree with an independent naive closure computation in all 500 comparisons. Three malformed Horn proof controls are rejected. No part of this experiment models `Lean`’s E-matching or current grind premise ranking.

13.7 What the measurements support

The measurements support four engineering conclusions. Exact replay can cleanly separate search from acceptance. Nonlinear representations complement each other in concrete examples. Recurrence-directed accumulator generalization makes a measurable difference to a small induction prover. Relevance slicing can preserve derivability while reducing work in a specified finite fragment.

They do not support a percentage improvement on Mathlib, a claim of faster kernel checking, or a claim that the complete `Forge` architecture has been implemented. Those questions require the `Lean` integration and comparative evaluation described next.

14 A credible evaluation protocol

14.1 Baselines and budgets

A future comparative suite should include stock grind, grind with the same explicitly retrieved lemmas, Aesop with a controlled grind leaf rule, Mathlib’s relevant specialized tactics, LeanHammer, and `Forge` with individual engines disabled. The equal-lemma baseline is essential: otherwise improved retrieval can be mislabeled as a superior algebraic algorithm.

Run two budget protocols. In the equal-total protocol, all tactics receive the same wall-clock limit and recorded CPU/resource policy, with reification and replay included. In the baseline-preserving protocol, reserve an unmodified grind allowance and report the additional allowance consumed by extensions. `Lean` heartbeats help control elaborator work but do not account for an external process’s CPU usage; record both.

Success means complete accepted proof under the specified axiom policy. Report unsolved goals, rejected reconstruction, unsupported fragments, and timeout separately. Count a proof with one residual goal as unsolved. Compile a fresh replay artifact, inspect its transitive axiom dependencies, and record declaration checking cost. A worker’s Boolean result is never the benchmark’s success oracle.

14.2 Corpus design

Separate development and held-out projects, not just randomly interleaved neighboring declarations. Existing lemmas in an imported module can leak the very theorem being tested, so generate each benchmark at the correct declaration point or explicitly restrict the environment. Record imported modules, source revision, theorem statement, available local hypotheses, and permitted external theorem supply.

Use strata that correspond to the design: ordinary grind regression goals; nonlinear inequalities with and without bounds; recursive data-structure specifications; accumulator and functional-induction goals; quantified lemmas with arithmetic side conditions; existential witnesses; and Boolean/finite-domain tasks under each trust policy. Include invalid goals and adversarial certificates as negative controls.

Do not choose only problems generated from a favored certificate language. Include sparse and dense polynomials, near-zero and exactly-zero boundary behavior, dependent indices, irrelevant assumptions, misleading induction variables, and goals requiring lemmas outside the first retrieval batch. Report theorem size and certificate size, not just success counts.

14.3 Ablations and acceptance gates

Disable exact quadratic decomposition, hypothesis products, subdivision, demand-directed premises, motive generalization, witness synthesis, and proof-cost-aware scheduling one at a time. Compare both total accepted proofs and paired wins/losses. A 10-point gain in one category does not excuse an undisclosed regression in the baseline category.

Before declaring the tactic significantly more capable, require a reproducible gain on held-out tasks at a fixed-total budget, plus no unexplained false successes or axiom-policy violations. Publish the problem list, pinned revisions, scripts, per-goal outcomes, replay artifacts, and resource accounting. This report does not attach an invented target success percentage; the first integrated benchmark should establish the baseline before a numeric performance objective is chosen.

15 Implementation roadmap and explicit engineering tasks

15.1 Stage 1: a narrow complete evidence path

Implement a standalone `Lean` tactic for rational-coefficient quadratic nonnegativity over $\mathbb{R}$. Reify the expression with an interpretation proof, run exact pivoting in `Lean` or import its certificate, reconstruct the weighted squares, and close the identity with a proved normalizer. Add a strict dependency audit and negative tests for malformed certificates, stale hypotheses, coefficient changes, and unsupported carriers.

This is the best first milestone because it crosses the entire boundary from a real **Lean** goal to a checked proof while keeping the certificate language small. Merely porting all search algorithms without finishing one acceptance path would create a large unvalidated prototype.

Next add finite-cone certificates with explicit hypothesis references. Search can remain out of process initially, but communication must use a bounded rational schema. Support no equality-ideal multipliers until their reconstruction path is tested. Compare direct proof terms with a proved reflective checker on coefficient identities; choose based on measured checking cost, not assumption.

15.2 Stage 2: obligations and induction

Implement the scoped obligation graph on top of Aesop or a small deterministic planner. Add lemma-application edges and route nonnegative polynomial demands to the validated solver. Use an adapter around grind rather than modifying its internals broadly. Record branch assumptions and reject stale solver replies.

Port the recurrence-argument analysis, then support structural induction with dependency-closed generalization in real **Lean** contexts. Test the eight supplied model examples and a separate held-out development using actual standard-library functions. Extend to function induction only after checking how generated recursors and equation lemmas are exposed in the pinned compiler.

At this stage a user should be able to solve a goal combining a recursive specification with a polynomial side condition and inspect a replay script containing both steps. Automatic invention of arbitrary invariants is not a prerequisite for this milestone.

15.3 Stage 3: bounded domains and selected external adapters

Formalize the Bernstein leaf theorem and a checked coverage tree, then connect bound demands to the planner. Begin with rational boxes over $\mathbb{R}$ and a fixed dimension/degree policy. Add exact coefficient expansion, strictness tags, and resource caps before optimization. Reuse the existing Python certificates as cross-language fixtures, but do not trust their labels.

Integrate one external prover through the common candidate/conditional-result API. Lean-SMT is useful for testing residual obligations; Duper is useful for native proof reconstruction. Add SAT in a clearly labeled trust mode, not as an unexamined dependency. Add witness construction for a limited, documented affine grammar before attempting general synthesis.

15.4 Stage 4: performance and richer certificates

Only after the held-out benchmark exists should the system optimize persistent indices, shared solver state, batched reification, proof DAG compression, SDP-based SOS search, equality-ideal multipliers, and learned scheduling. Each optimization must preserve the same receipt semantics and be tested against the unoptimized acceptance layer.

A practical module layout is:

```

Forge/Core/Scope.lean      -- context identity and transfer
Forge/Core/Obligation.lean -- AND/OR graph and constructors
Forge/Core/Accept.lean     -- reconstruction and axiom policy
Forge/Adapters/Grind.lean  -- version-specific native interface
Forge/Arith/Reify.lean     -- expression interpretation proofs
Forge/Arith/Quadratic.lean -- exact rational decomposition
Forge/Arith/Cone.lean      -- certificate theorem and replay
Forge/Arith/Bernstein.lean -- box identity and coverage theorem
Forge/Search/Demand.lean   -- typed premise obligations
Forge/Search/Induction.lean -- recurrence/motive analysis
Forge/Search/Witness.lean  -- bounded symbolic witnesses
Forge/Adapters/External.lean -- proof-producing backend protocol
Forge/Replay.lean          -- deterministic receipt emission
Forge/Tactic.lean          -- parser and user-facing command

```

These paths describe proposed production modules; they are not files claimed to exist in the accompanying prototype.

16 Failure modes and safeguards

Failure mode	Why it is dangerous	Required safeguard
Floating residual treated as zero	A tiny negative constant can invalidate a universal inequality	Exact coefficient identity and exact sign checks
Branch hypothesis leaks	A conditional statement becomes unconditional	Scope dependencies and checked implication abstraction
Wrong carrier or cast	Real, integer, natural, and modular operations differ	Typed reification and explicit transport lemmas
Unsafe denominator clearing	Sign reversal or a zero branch is lost	Generate and prove sign/nonzero obligations
Unjustified induction generalization	The step uses a stronger hypothesis than the motive permits	Reconstruct the motive and recursor application
Induction at the original input	The proof is circular	Fix the predecessor/tail; use genuine well-founded evidence
Uncovered domain cells	Local positivity is reported as global	Checker derives children from each parent cut
Strictness is erased	Zero is accepted as positive	Separate nonnegative and positive certificate rules
External proof holes ignored	Backend unsatisfiability is mislabeled a theorem	Every residual obligation remains an AND child
Native evaluation hidden	Claimed trust differs from actual dependencies	Explicit policy and transitive axiom inspection
Opaque countermodel misread	An abstraction model is called a real counterexample	Validate its original semantics and assumptions
Certificate causes resource exhaustion	An attacker supplies enormous exponents or trees	Byte, node, degree, coefficient-size and replay budgets

Failure mode	Why it is dangerous	Required safeguard
Cache uses a printed goal	Different instances or contexts collide	Elaboration-aware key plus rechecking before assignment
Heuristic prunes every useful route	A theoretically useful solver becomes brittle	Fair fallback and clearly reported bounded incompleteness

The prototype contains some size checks and negative tests, but it is not a hardened untrusted-input service. In production, parse lengths before allocating large arrays, cap polynomial expansion before multiplying enormous supports, and treat interrupts and out-of-memory behavior as failures rather than partial successes. Operational reliability is separate from mathematical certificate soundness.

17 Conclusion

The strongest practical successor to `grind` should not discard it. The proposed **Forge** uses its native equality and theory infrastructure as a foundation, then adds the ability to plan and prove missing premises, search for nonlinear order certificates, choose useful induction motives, and construct witnesses. Those additions address proof tasks that cannot be reduced to simply increasing the same saturation budget.

The concrete starting point is the exact evidence path. Rational quadratic decomposition, finite polynomial cones, Bernstein subdivision, and accumulator-aware induction have explicit algorithms and small mathematical acceptance rules. Their cooperation can be expressed with scoped proof obligations without trusting numerical optimizers, theorem rankers, or search schedulers. The package demonstrates these mechanisms and also records failures that prevent overstating their scope.

The next decisive result is not another architecture diagram: it is a compiled **Lean** implementation of one complete certificate path, followed by a held-out, equal-budget comparison with current `grind` and the existing automation ecosystem. Until that experiment is performed, the justified claim is a concrete design with tested mechanisms and a credible route to substantially broader automation, not an already measured universal replacement.

A Package contents and reproduction

The archive contains the article’s $\text{\LaTeX}$ source and compiled PDF, eight Python source files for search and replay, row-level CSV results, the JSON summary, 24 certificate bundles, generated **Lean** arithmetic proofs, induction examples, a proposed data-contract sketch, and the pinned example-project configuration. It does not contain a production forge tactic, a **Lean** toolchain, third-party font files, or externally fetched proof libraries.

From the package root, reproduce the Python mechanisms with:

```
python -m pip install -r requirements.txt
python prototype/run_all.py
python prototype/replay.py results/certificates/*.json
```

The second command reruns the experiments and regenerates their results. The third command replays the saved certificate bundles without importing NumPy, SciPy, or SymPy. On shells without glob expansion, pass filenames explicitly or use a shell that expands the wildcard.

The compiler-facing examples require a local **Lean** installation and fetched Mathlib dependencies. In the supplied `lean/` directory, the intended setup is:

```
lake update
lake exe cache get
```

Then, from the package root:

```
python scripts/check_lean.py
```

These installation and compilation commands were not executed successfully in the supplied experiment environment. The runner records missing tools, compilation errors, and timeouts separately. Its compilation result is not a full axiom-policy audit or a comparison against `grind`.

Build the article from its directory using XeLaTeX, with installed STIX and DejaVu Sans Mono fonts:

```
xelatex -interaction=nonstopmode -halt-on-error forge.tex
xelatex -interaction=nonstopmode -halt-on-error forge.tex
```

No font files are included in the archive. The README gives the same commands and summarizes the evidence boundaries.

B Certificate schema and replay invariants

B.1 Polynomial and cone objects

A polynomial is a variable count and a list of exponent-vector/rational-coefficient pairs. Coefficients are strings such as `"3/7"`, never JSON floating-point numbers. The parser validates exponent shape and builds a canonical sparse representation. The theorem's variable interpretation belongs to the **Lean** adapter, not to an implicit convention in a string variable name.

A cone term records a nonnegative rational weight, the polynomial to square, and a list of indices into the *supplied* nonnegative hypotheses. A bundle includes the target and the complete hypothesis polynomials. Replay checks the exact identity. The certificate is valid *under those hypotheses*; replay does not establish that the hypotheses hold at any particular point.

The **Lean** adapter must therefore map each hypothesis index to an actual proof of the corresponding reified inequality, not just copy the polynomial list into the checker. The reification equalities connect the certificate's arithmetic syntax to the original terms.

B.2 Bernstein objects

A root bundle supplies the target polynomial, the rational box, a strictness flag, and the tree. Leaves contain degree vectors and coefficient vectors. Splits contain only an axis, cut, and child certificates. Child domains are computed during replay, so a producer cannot omit a strip of the domain by supplying inconsistent endpoints.

A negative Bernstein coefficient is not a negative value of the polynomial. It is only failure of the sufficient certificate condition. Conversely, a positive sampled value is not enough to accept a leaf. The identity and all coefficient bounds are required.

B.3 Equational-induction objects

An induction bundle is a sequence of theorem statements and receipts. Each term records its symbol, sort, arguments, and variable status. A rewrite step records a location, a named admitted defining equation or previously checked lemma, and the resulting term. The checker reconstructs the substitution and replacement rather than trusting the recorded output.

The base and step goals are derived from the theorem and the selected induction subject. The checker installs no lemma until the receipt succeeds. The type universe is intentionally limited to the prototype's list, element, and natural-number sorts; this format is not a serialization of arbitrary `Lean` expressions.

B.4 Horn objects

A Horn bundle supplies initial facts, the ordered rule list, the target, and a topologically ordered list of rule applications. Replay checks that every referenced rule exists, that its premises are already known, and that the recorded conclusion is the rule's actual conclusion. The target must be in the resulting closure.

These schemas make the computations inspectable and replayable. A production format should add explicit versioning, canonical encoding, resource limits, and an environment fingerprint, while preserving the principle that every semantic assumption is supplied as a checked premise.

C Selected implementation excerpts

C.1 Exact quadratic pivoting

The following is the core loop from the executed quadratic prototype, with surrounding input construction omitted. Here H is an exact rational matrix and z the affine monomial basis.

```
while any(c for row in H for c in row):
    if any(H[i][i] < 0 for i in range(size)):
        return None, {'status': 'unknown_non_psd'}
    k = next((i for i in range(size) if H[i][i] > 0), None)
    if k is None:
        return None, {'status': 'unknown_non_psd'}
    d = H[k][k]
    v = H[k][:]
    s = sum((a*b for a, b in zip(v, z)), Poly.const(p.n, 0))
    out.append((1/d, Atom(s)))
    H = [[H[i][j] - v[i]*v[j]/d
           for j in range(size)] for i in range(size)]
certificate = ConeCertificate(tuple(out))
return certificate if check_cone(p, (), certificate) else None
```

The excerpt abbreviates the function's statistics return values; the executable file is authoritative. The exact identity checker is still used even though the producer itself uses exact arithmetic.

C.2 The acceptance principle for cone terms

```
total = Poly.const(target.n, 0)
for weight, atom in certificate.terms:
```

```

    if not isinstance(weight, Fraction) or weight < 0:
        return False
    if atom.square.n != target.n:
        return False
    total += weight * atom.expand(hypotheses)
return total == target

```

Hypothesis reference validation occurs inside `atom.expand`; exception handling and arity checks are present in the complete file. There is intentionally no epsilon, numerical norm, or solver-status test in this acceptance condition.

C.3 The induction distinction in **Lean** syntax

This is the essential proof shape supplied in `InductionExamples.lean`, not a claim of a compiled example or an implemented forge command:

```

theorem reverseAcc_spec (xs acc : List  $\alpha$ ) :
  reverseAcc xs acc = xs.reverse ++ acc := by
  induction xs generalizing acc with
  | nil => simp [reverseAcc]
  | cons x xs ih =>
    simp [reverseAcc, List.reverse_cons, ih, List.append_assoc]

```

The mathematical content of the design's induction extension lies in discovering the appropriate generalization and producing a valid motive, not in introducing a new kernel rule.

Sources

- [1] Lean FRO. *Lean 4.34.0 release*. September 14, 2026.
<https://github.com/leanprover/lean4/releases/tag/v4.34.0>.
- [2] The Mathlib Community. *Mathlib toolchain at commit 1cf325a0cf67aca2b04d76b5380ff6a9e410aefa*. Accessed September 14, 2026, Pacific time. <https://github.com/leanprover-community/mathlib4/blob/1cf325a0cf67aca2b04d76b5380ff6a9e410aefa/lean-toolchain>.
- [3] Lean FRO. *The Lean Language Reference: The grind tactic*. Version 4.34.0 as served at access.
<https://lean-lang.org/doc/reference/latest/The--grind--tactic/>.
- [4] Lean FRO. *Algebraic Solver (Commutative Rings, Fields)*. https://lean-lang.org/doc/reference/latest/The--grind--tactic/Algebraic-Solver-_LPAR_Commutative-Rings___-Fields_RPAR_.
- [5] Lean FRO. *Linear Arithmetic Solver*. <https://lean-lang.org/doc/reference/latest/The--grind--tactic/Linear-Arithmetic-Solver/>.
- [6] Lean FRO. *E-matching*.
https://lean-lang.org/doc/reference/latest/The--grind--tactic/E___matching/.
- [7] Leonardo de Moura and Lean contributors. *Lean.Meta.Tactic.Grind.Action*. Lean source, tag v4.34.0.
<https://github.com/leanprover/lean4/blob/v4.34.0/src/Lean/Meta/Tactic/Grind/Action.lean>.
- [8] Kim Morrison and Leonardo de Moura. *grind: An SMT-Inspired Tactic for Lean 4 (Short Paper–System Description)*. IJCAR 2026, LNCS 16688.
https://doi.org/10.1007/978-3-032-32589-1_7.
- [9] The Aesop contributors. *Aesop: White-box automation for Lean*. Source and documentation.
<https://github.com/leanprover-community/aesop>.
- [10] Joshua Clune and contributors. *LeanHammer*. Source and documentation.
<https://github.com/JOSHCLUNE/LeanHammer>. See also Thomas Zhu et al., *Premise Selection for a Lean Hammer*, 2025, <https://arxiv.org/abs/2506.07477>.
- [11] Evan Wang, Simon Chess, Sophie Szeto, and Theodore Meek. *Learned Interventions in Lean 4 grind*. 2026. <https://arxiv.org/abs/2607.22972>.
- [12] The Mathlib Community. *Mathlib.Tactic.Linarith.Frontend*. https://leanprover-community.github.io/mathlib4_docs/Mathlib/Tactic/Linarith/Frontend.html.
- [13] The Mathlib Community. *Mathlib.Tactic.LinearCombination*. https://leanprover-community.github.io/mathlib4_docs/Mathlib/Tactic/LinearCombination.html.
- [14] The Duper contributors. *Duper*. Source and documentation.
<https://github.com/leanprover-community/duper>.
- [15] The Lean-auto contributors. *Lean-auto*. Source and documentation.
<https://github.com/leanprover-community/lean-auto>. See also *Lean-auto: An Interface between Lean 4 and Automated Theorem Provers*, 2025, <https://arxiv.org/abs/2505.14929>.

- [16] The Lean-SMT contributors. *Lean-SMT*. Source and documentation.
<https://github.com/ufmg-smite/lean-smt>. See also *lean-smt: An SMT Tactic for Discharging Proof Goals in Lean*, 2025, <https://arxiv.org/abs/2505.15796>.
- [17] Marcus Rossel and contributors. *Lean-egg*. Repository and deprecation notice.
<https://github.com/marcusrossel/lean-egg>.
- [18] Max Willsey et al. *egg: Fast and Extensible Equality Saturation*. 2020/2021.
<https://arxiv.org/abs/2004.03082>.
- [19] Lean FRO. *The Lean Language Reference: Tactic Reference*. Particularly SAT Solver Integration, Inductive Types, and Call-by-Value Evaluation.
<https://lean-lang.org/doc/reference/latest/Tactic-Proofs/Tactic-Reference/>.
- [20] John Harrison. *Verifying nonlinear real formulas via sums of squares*. TPHOLs 2007, LNCS 4732, pp. 102–118. Author’s publication page: <https://www.cl.cam.ac.uk/~jrh13/papers/sos.html>.
- [21] The SciPy contributors. *scipy.optimize.linprog*. API documentation.
<https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.linprog.html>. The supplied experiment used SciPy 1.17.0.
- [22] NASA Langley Formal Methods Program. *PVS Bernstein Library*.
<https://shemesh.larc.nasa.gov/fm/pvs/Bernstein/>.

Online sources were checked on September 14, 2026, Pacific time. Moving documentation URLs may change; compiler-facing example files are pinned separately. Original algorithms, proof arguments, and prototype measurements in this report are distinguished from the cited descriptions of existing software.